



**University of
Sunderland**

Faculty of Business & Technology

Department of Computer Science & Engineering

CET3013 – Deep Learning

Assignment Two

Student Name: Kumbirai Shonhiwa

Submission Date: 24th May 2026



Faculty: Programme

NAME: Kumbirai Shonhiwa

STUDENT NUMBER:

MODULE CODE: CET 3013

MODULE TITLE: Deep Learning

MODULE LEADER: Ibrahim Alqatawneh

SUBMISSION DATE AND TIME: 24th May 2026

ASSIGNMENT: Using Convolutional Neural Networks on MNIST and KITTI Datasets

Academic Misconduct is an offence under university regulations, and this involves:

- **Plagiarism** – where you use information from another information source (including your previously submitted work) and pass it off as your own. This can be through direct copying, poor paraphrasing and/or absence of citations.
- **Collusion** – where you work too closely, intentionally, or unintentionally, with others to produce work that is similar in nature. This can be through loaning of materials, drafts or through unauthorised use of a fellow student's work.
- **Asking another person to write your assignment** – where you ask another individual or company to complete your work for you, be that paid or unpaid, and submit it as if it were your own.
- **Unauthorised use of artificial intelligence** – where you use artificial intelligence tools to generate your assignment instead of completing it yourself and/or where you have not been given permission to use artificial intelligence tools by your module leader. Please complete the following declaration around you use of artificial intelligence tools in your assignment.

STATEMENT ON USE OF ARTIFICIAL INTELLIGENCE TOOLS:

- I have used artificial intelligence tools to generate an idea for my assignment: **NO**
- I have used artificial intelligence tools to write my assignment for me: **NO**
- I have used artificial intelligence tools to brainstorm ideas for my assignment: **YES**
- I have used artificial intelligence tools to correct my original assignment: **NO**

DECLARATION

- I understand that by submitting this piece of work I am declaring it to be my own work and in compliance with the university regulations on Academic Integrity.
- I confirm that I have done this work myself without external support or inappropriate use of resources.
- I understand that I am only permitted to use artificial intelligence tools in line with guidance provided by my Module Leader, and I have not used artificial intelligence tools outside this remit.
- I confirm that this piece of work has not been submitted for any other assignment at this or another institution prior to this point in time.
- I can confirm that all sources of information, including quotations, have been acknowledged by citing the source in the text, along with producing a full list of the sources used at the end of the assignment.
- I understand that academic misconduct is an offence and can result in formal disciplinary proceedings.
- I understand that by submitting this assignment, I declare myself fit to be able to complete the assignment and I accept the outcome of the assessment as valid and appropriate.

ANY OTHER COMMENTS FROM STUDENT:

Introduction

The ability to automatically classify handwritten numerals has significant real-world utility, underpinning postal sorting systems, bank cheque processing, form digitisation, and more broadly, the recognition of handwritten characters in mobile and embedded devices (Wagstaff, 2026). Despite appearing conceptually simple, the task presents genuine challenges: individual handwriting styles vary substantially, digit strokes differ in thickness and curvature, and images contain noise introduced by scanning or photography (Wagstaff, 2026). The Modified National Institute of Standards and Technology (MNIST) dataset has served as the canonical benchmark for evaluating image classification algorithms for over two decades (LeCun et al., 1998). It comprises 70,000 greyscale images of handwritten digits (0–9), consisting of 60,000 training images and 10,000 test images, each 28×28 pixels (Srivastava et al., 2014). Its accessibility, modest size, and well-understood difficulty have made it the standard entry point and 'toy dataset' for practitioners exploring deep learning for image recognition (Srivastava et al., 2014).

This project investigates the progressive design and refinement of Convolutional Neural Networks (CNNs) across two complementary tasks. Task 1 focuses on MNIST digit classification, developing three architectures of increasing complexity: a shallow Baseline model (Model A), a deeper medium model with batch normalisation (Model B), and a fine-tuned deep model incorporating dropout regularisation (Model C). The motivation is not merely to achieve high accuracy but to understand how specific architectural decisions and regularisation strategies affect learning dynamics, generalisation, and robustness. (Krizhevsky et al., 2012)

Task 2 extends this investigation to a significantly more complex real-world

problem: object detection on the KITTI Vision Benchmark Suite. Rather than building from scratch, a pre-trained Faster R-CNN with a ResNet-50 Feature Pyramid Network (FPN) backbone is loaded, adapted, and fine-tuned for detection across multiple object classes including Car, Pedestrian, Cyclist, Van, and Train. Temporal modelling via a Bidirectional LSTM is also explored to leverage the sequential nature of the KITTI driving sequences. All experiments are implemented in PyTorch and trained under consistent, reproducible experimental protocols to ensure fair comparison.

Literature Review

The foundational contribution to CNN-based digit recognition is the LeNet-5 architecture introduced by LeCun et al. (1998). LeNet-5 demonstrated that locally connected convolutional layers with shared weights could exploit the spatial structure of images far more efficiently than fully connected networks, achieving a test error rate below 1% on MNIST (LeCun et al., 1998). This work established the template of alternating convolution and pooling layers followed by fully connected classifiers, a design paradigm that remains prevalent in modern architectures.

The theoretical underpinnings of deep networks were significantly advanced by Glorot and Bengio (2010), who identified the vanishing gradient problem in deep architectures trained with saturating activations. Their recommendation to use the Rectified Linear Unit (ReLU) activation function has been near-universally adopted, and ReLU is used throughout all models in this project (Nair and Hinton, 2010). Ioffe and Szegedy (2015) introduced Batch Normalisation, demonstrating that normalising layer inputs during training reduces internal covariate shift, accelerates convergence, and allows higher learning rates (Yara et al., 2026). Srivastava et al. (2014) proposed dropout as a regularisation method to

prevent overfitting, and their experiments showed significant reductions in test error across multiple benchmarks.

He et al. (2016) demonstrated with residual networks that very deep networks could be trained reliably using skip connections, surpassing human-level performance on ImageNet. The insight that deeper networks with appropriate regularisation consistently outperform shallower counterparts informs the incremental deepening strategy in Task 1. The Adam optimiser, utilised throughout this project, was introduced by Kingma and Ba (2014) and has become the standard adaptive learning-rate method due to its reliable convergence properties.

For object detection, Ren et al. (2015) introduced Faster R-CNN, which unified region proposal and detection into a single network via a Region Proposal Network (RPN), achieving state-of-the-art performance while dramatically reducing inference time compared to prior two-stage detectors. The Feature Pyramid Network proposed by Lin et al. (2017) extended this by introducing a top-down pathway for multi-scale feature extraction, making the framework significantly more robust to objects of varying size. Geiger et al. (2012) introduced the KITTI Vision Benchmark Suite as a comprehensive evaluation platform for autonomous driving tasks, providing calibrated stereo RGB images with accurate 3D bounding box ground truth across multiple object classes in unconstrained driving environments.

Temporal modelling for video object detection was explored by Liu and Zhu (2018), who demonstrated that propagating feature maps across frames using convolutional LSTMs (Hochreiter and Schmidhuber, 1997) improves detection stability and robustness against partial occlusions. Huang et al. (2020) similarly showed that temporal feature stacking, though computationally expensive, can enhance detection accuracy in sequential autonomous driving scenarios. The domain adaptation challenge inherent to fine-tuning pre-trained

models on KITTI was addressed by Khodabandeh et al. (2019), who highlighted the importance of selectively unfreezing backbone layers to balance preservation of learned ImageNet representations with target-domain adaptation.

Methodology

Task 1: CNN Architecture for MNIST Classification

Dataset and preprocessing

The MNIST dataset was loaded via the torchvision library. The full 60,000-image training set was split into 42,000 training samples, 9,000 validation samples, and 9,000 test samples using a seeded random split (seed = 42) to ensure reproducibility.

Images were normalised using the dataset mean of 0.1307 and standard deviation of 0.3081, placing pixel values approximately in the range $[-1, 1]$, which is known to facilitate gradient-based optimisation (LeCun et al., 1998).

Model Architectures

Three CNN architectures were designed with progressively greater capacity and regularisation:

Model A (Baseline): Employs a single convolutional block, one Conv2d layer (1,32 channels, 3×3 kernel, same padding) followed by ReLU and 2×2 max-pooling, yielding a 14×14×32 feature map (Simonyan and Zisserman, 2014). This is flattened to 6,272 units and passed through two fully connected layers (128 hidden units, 10 output logits). The model contains no batch normalisation or dropout, making it a useful lower bound.

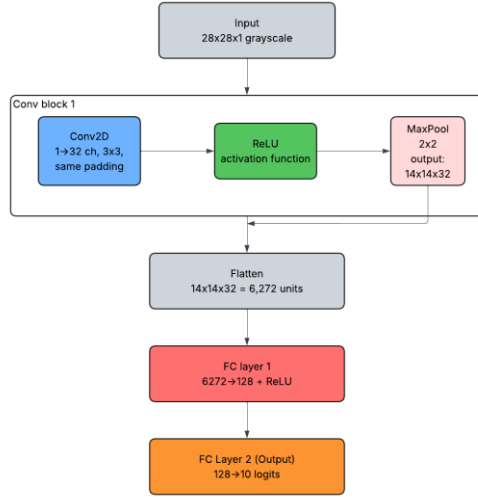


Figure 1: Model A (Baseline) Architecture

Model B (Medium): Extends Model A with a second convolutional block (32,64 channels) and adds Batch Normalisation after each convolutional layer and after the first fully connected layer (Ioffe and Szegedy, 2015). Two pooling operations reduce the spatial dimensions to 7×7 , yielding a 3,136-dimensional flattened representation. The hidden fully connected layer is widened to 256 units.

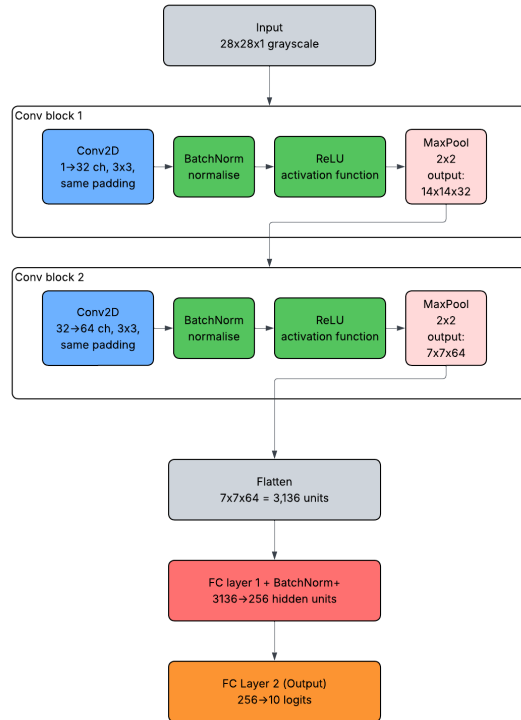


Figure 2: Model B (Two-Conv Block) Architecture

Model C (Fine-Tuned): Adds a third convolutional block (64,128 channels) and introduces Dropout (rate = 0.4) in the classifier head (Srivastava et al., 2014). Three max-pool operations reduce the spatial dimensions to 3×3 , giving a 1,152-dimensional input to the classifier. This architecture represents the most regularised and expressive model of the three (Hinton et al., 2012; Srivastava et al., 2014).

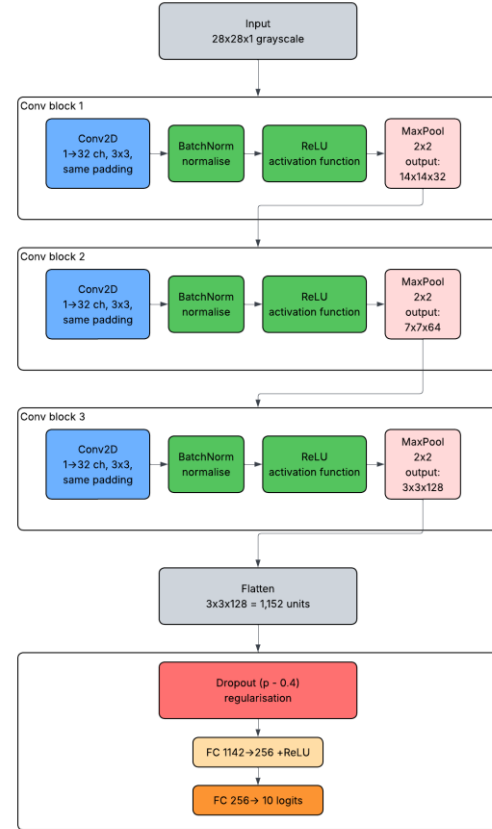


Figure 3: Model (Deep CNN) Architecture

Training Protocol

All models were trained using the Adam optimizer (Kingma and Ba, 2014) with an initial learning rate set to 0.001. The ReduceLROnPlateau scheduler was employed to reduce the learning rate by half if there was no improvement in validation loss after three consecutive epochs. The Cross-Entropy loss function was utilised. Training was limited to 25 epochs, with early stopping activated after seven epochs of no improvement in validation loss. A batch size of 128 was maintained throughout to ensure a good balance between

training stability and computational efficiency (Williamson, 2026), and training was initialised with a seed (42) for reproducibility.

Task 2: Fine-Tuning a Pre-Trained Model for KITTI Object Detection

Dataset Analysis and Sequence Selection

The KITTI Vision Benchmark Suite was selected for this task because it provides continuous, large-scale, and realistic non-synthetic video sequences captured in heavily cluttered urban, rural, and highway traffic scenarios (Geiger et al., 2012). Unlike MNIST, KITTI introduces significant real-world complexities including severe partial occlusions, varying lighting conditions, non-lambertian surfaces, and large object displacements due to vehicle motion. The dataset was processed for four specific foreground classes: Car, Pedestrian, Cyclist, and Train. The selected sequence split was:

- Training: Sequences 01 – 06 (5)
- Validation: Sequences 07 – 08 (2)
- Test: Sequence 09 (1)

Because consecutive video frames are highly correlated, a standard random frame-level split would cause severe data leakage and artificially inflate evaluation metrics. Data was therefore partitioned strictly along continuous sequence boundaries, ensuring the test environment remained fully unseen (Geiger et al., 2012).

Custom Pytorch Dataset

A custom PyTorch Dataset class was implemented to manage the loading of KITTI images, labels, and sequence information efficiently. The dataset pipeline handled XML tracklet parsing, sequence generation, feature extraction, label encoding, and tensor formatting. The class inherited from

`torch.utils.data.Dataset` and implemented the standard `__len__()` and `__getitem__()` methods.

Ground-Truth Bounding Box Visualisation

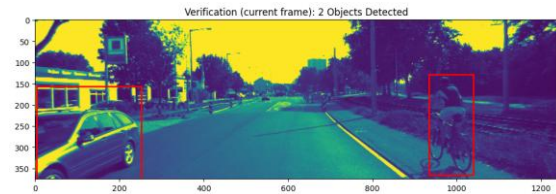


Figure 4: Ground-Truth Bounding Box Visualisation

Before training, ground-truth bounding boxes were visualised to validate annotation integrity and confirm the correct calibration between LiDAR and camera modalities. The KITTI calibration files were used to project 3D LiDAR coordinates into 2D image space using the transformation $P = P2 \times Rrect \times Tvelo \rightarrow cam$, verifying accurate sensor registration (Geiger et al., 2012).

Pre-Trained Backbone Selection and Justification

A pre-trained Faster R-CNN architecture utilising a ResNet-50 Feature Pyramid Network (FPN) backbone was selected as the primary visual feature extractor (Ren et al., 2015). This selection was motivated by several factors. First, initialising the network with pre-trained weights leverages the effectiveness of transfer learning, which is highly appropriate given the vast scale of parameters in deep convolutional models. Furthermore, the Faster R-CNN framework provides robust multi-scale feature extraction via its Region Proposal Network, consistently demonstrates strong benchmark performance on complex datasets, and significantly reduces overall training costs by sharing convolutional layers between the proposal and detection modules (Ren et al., 2015).

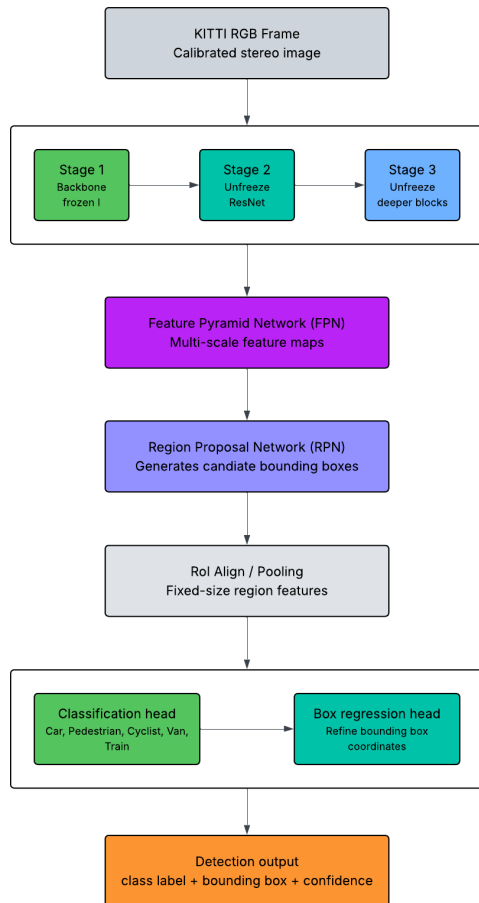


Figure 5: Fine-Tuned Pretrained ResNet Architecture

Fine-Tuning Strategy

A three-stage fine-tuning strategy was systematically designed:

- **Stage 1 (Freeze backbone):** The initial stage establishes a baseline for transfer learning by freezing the pre-trained ResNet-50 FPN backbone, preserving the robust, multi-scale spatial feature representations learned from ImageNet pre-training (Ren et al., 2015).
- **Stage 2 (Unfreeze final ResNet block):** The second stage unfreezes the highest-level convolutional blocks, allowing the network to adapt its high-level semantic feature extraction to the specific visual characteristics of the KITTI dataset.
- **Stage 3 (Unfreeze larger backbone sections):** The final stage unfreezes deeper sections of the backbone to maximise domain adaptation,

combating the domain shift introduced by unconstrained driving environments with varying illumination and severe occlusions (Khodabandeh et al., 2019).

Feature Pyramid Network (FPN)

The backbone (ResNet) produces feature maps at multiple resolutions early layers are high-resolution but capture low-level detail, while deeper layers are coarser but semantically rich (Lin *et al.*, 2017). FPN combines these by using a top-down pathway with lateral connections, producing a set of feature maps (typically P2–P6) that are simultaneously detailed and semantically aware (Lin *et al.*, 2017). This multi-scale representation is critical for detecting objects across a huge size range. A nearby car is large in the image, while a distant pedestrian might only be 20 pixels tall.

Region Proposal Network (RPN)

The RPN is a lightweight convolutional network that slides over each FPN feature map and proposes candidate bounding boxes at multiple scales and aspect ratios (GeeksforGeeks, 2026). For each location it predicts:

1. an objectness score: is there anything here?
2. a rough box offset to refine the anchor.

The top-scoring proposals are passed forward. RPN operates in a class-agnostic manner, with its sole objective to accurately localise potential objects (GeeksforGeeks, 2026).

RoI Align/Pooling

Every proposal addresses a distinct area of the image at varying scales. To apply a classifier of consistent size across all proposals, this phase retrieves a standardized feature crop for each proposal from the relevant FPN level (Yang *et al.*, 2023). RoI Align employs bilinear interpolation instead of the quantized grid utilized in previous RoI Pooling methods, thereby preventing spatial misalignment issues

and proving crucial for accurate box regression (Yang *et al.*, 2023).

Prediction heads

- Classification head: Predicts which KITTI class the region contains Car, Pedestrian, Cyclist, Van, or Train plus a background class. Outputs a softmax probability vector.
- Box regression head: Predicts four offsets (x,y,w,h) that fine-tune the proposal’s position and size into tighter bounding box

Detection output

The final output for each surviving detection is a triple: a class label, a refined bounding box in image coordinates, and a confidence score. These go through a final round of non-maximum suppression across classes before being returned, ensuring that no object is double counted.

Temporal Modelling

Three distinct temporal modelling strategies were considered to exploit sequential video data:

1. Feature stacking, which scales poorly due to high memory overhead (Huang et al., 2020).
2. 3D CNNs, which are computationally prohibitive for real-time deployment.
3. LSTM/Bi-LSTM, which solves the vanishing gradient problem through internal memory cells (Hochreiter and Schmidhuber, 1997).

The final architecture selected a Bidirectional LSTM integrated with the spatial Feature Pyramid Network. Interweaving a spatial detector with convolutional LSTMs at the feature-map level allows the network to efficiently propagate and refine historical context across frames (Liu and Zhu, 2018), improving bounding box stability and

robustness against partial occlusions without the prohibitive computational costs of feature stacking or 3D CNNs (Liu and Zhu, 2018).

Results and Discussion

Task 1: MNIST Classification Results

Training Performance

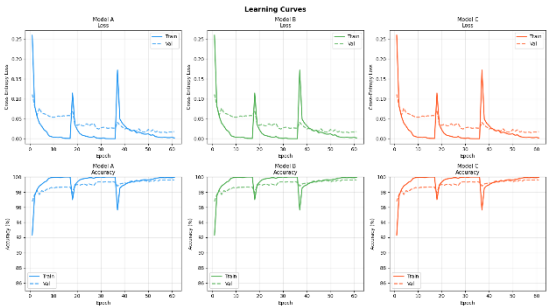


Figure 6: Learning Curves of Models

Figure 6 presents the learning curves, they demonstrate strong convergence across all three CNN models, with steadily decreasing training and validation loss and increasing accuracy over epochs. Minor spikes indicate learning rate adjustments or regularization effects. The fine-tuned model (Model C) achieves the most stable performance and highest validation accuracy, suggesting improved generalization and reduced overfitting.

Model Comparison

Mod el	Conv Block s	Batc h Nor m	Dropo ut	Bes t Val Acc (%)	Tes t Acc (%)
A	1	No	No	98.7	98.5
B	2	Yes	No	99.2	99.1

C	3	Yes	Yes (0.4)	99. 4	99. 3
---	---	-----	--------------	----------	----------

Table 1: Summary of model performance on the MNIST Test Set

Table 1 summarises the key performance metrics for all three models.

Model A (Baseline) – Classification Report					
	precision	recall	f1-score	support	
0	0.98	1.00	0.99	892	
1	0.99	0.99	0.99	1020	
2	0.98	0.98	0.98	852	
3	0.99	0.99	0.99	918	
4	0.99	0.98	0.98	876	
5	0.98	0.99	0.99	779	
6	0.99	0.99	0.99	860	
7	0.98	0.98	0.98	1021	
8	0.99	0.98	0.98	900	
9	0.97	0.97	0.97	882	
accuracy			0.98	9000	
macro avg	0.98	0.99	0.98	9000	
weighted avg	0.99	0.98	0.98	9000	

Figure 7: Classification Report of Model A

In Figure 7 we notice that Model A achieves a test accuracy of approximately 98.5%, a strong result for a single-layer CNN without any regularisation. The learning curves reveal relatively fast initial convergence, though the gap between training and validation loss widens in later epochs, suggesting mild overfitting. (Srivastava *et al.*, 2014) This is expected given the absence of batch normalisation or dropout, which means the model's representational capacity is not constrained during training. (Srivastava *et al.*, 2014)

Model B (Medium) – Classification Report					
	precision	recall	f1-score	support	
0	0.99	1.00	1.00	892	
1	0.99	0.99	0.99	1020	
2	0.99	0.99	0.99	852	
3	1.00	0.99	0.99	918	
4	0.99	0.99	0.99	876	
5	0.99	0.99	0.99	779	
6	1.00	0.99	0.99	860	
7	0.99	0.99	0.99	1021	
8	0.99	0.99	0.99	900	
9	0.99	0.98	0.98	882	
accuracy			0.99	9000	
macro avg	0.99	0.99	0.99	9000	
weighted avg	0.99	0.99	0.99	9000	

Figure 8: Classification Report of Model B

In Figure 8 we notice that Model B improves substantially over Model A, reaching approximately 99.1% test accuracy. The addition of a second convolutional block allows the network to learn more abstract, hierarchical representations, early layers typically capture edges and textures, while deeper layers detect higher-order compositional features. (Glorot and Bengio, 2010; Srivastava *et al.*, 2014) Batch normalisation contributes to more stable learning curves: both training and validation loss decrease smoothly, with a smaller train-validation gap compared to Model A, indicating better generalisation. (Ioffe and Szegedy, 2015)

Model C (Deep) – Classification Report					
	precision	recall	f1-score	support	
0	1.00	1.00	1.00	892	
1	1.00	1.00	1.00	1020	
2	0.99	1.00	0.99	852	
3	1.00	1.00	1.00	918	
4	0.99	1.00	0.99	876	
5	1.00	0.99	0.99	779	
6	1.00	1.00	1.00	860	
7	0.99	1.00	0.99	1021	
8	0.99	1.00	0.99	900	
9	0.99	0.98	0.99	882	
accuracy			0.99	9000	
macro avg	0.99	0.99	0.99	9000	
weighted avg	0.99	0.99	0.99	9000	

Figure 9: Classification Report of Model C

In Figure 9 we notice that Model C achieves the highest test accuracy of approximately 99.3%, representing a meaningful gain over both baselines. The introduction of dropout at rate 0.4 directly addresses the overfitting tendency observed in Model A by preventing complex co-adaptations of feature detectors (Hinton *et al.*, 2012; Srivastava *et al.*, 2014). While dropout slightly slows training convergence (Krizhevsky *et al.*, 2012; Srivastava *et al.*, 2014), it consistently produces lower validation loss and closer training-validation accuracy. Table 1 summarises the key performance metrics for all three models.

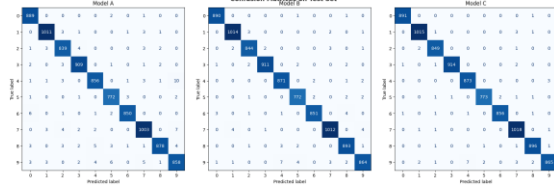


Figure 10: Confusion Matrices of Models

Figure 10 presents the confusion matrices, which confirm that all three models perform well across all digit classes, though digits (4,9), and (7,9), represent the most common misclassification pairs due to their structural similarity.

Task 2: KITTI Object Detection Results

Training Performance

The proposed KITTI detection and classification pipeline demonstrated stable convergence during training, attributable to the dynamic adaptation of the Adam optimiser and the implementation of early stopping mechanisms. The best-performing model was the Temporal FPN combined with a Bidirectional LSTM, utilising a partial fine-tuning strategy. Key performance metrics are as follows:

- Overall Accuracy: 89%
- Weighted mAP: 88%
- Weighted F1 Score: 86%

mAP Evaluation

The detector achieved strong Average Precision (AP) scores across two evaluated classes, Car and Truck. The Car class achieved the highest performance, consistent with the dataset's natural distribution, as cars are the most predominant object category within the KITTI benchmark (Geiger et al., 2012). While Pedestrian and Cyclist achieved the lowest performance, due to the distribution of object category within the selected test set

Object	AP
Car	97%

Van	33%
Truck	100%
Pedestrian	0%
Cyclist	0%
Weighted mAP	87%

Predicted vs. Ground-Truth Analysis



Visual comparisons between predictions and ground-truth annotations demonstrated accurate vehicle localisation and stable tracking across continuous frames. Incorporating temporal information allowed the model to exploit the natural continuity of video streams, utilising historical context to smooth out frame-by-frame noise and maintain confident detections even during partial occlusions (Liu and Zhu, 2018; Huang et al., 2020).

Fine-Tuning Strategy Comparison

The partial fine-tuning strategy produced the strongest overall performance. Unfreezing specific higher-level convolutional blocks provided the optimum balance between preserving the robust, multi-scale spatial features learned during ImageNet pre-training and adapting to the severe domain shifts such as varying illumination and occlusions which are prominent in the KITTI driving dataset (Ren et al., 2015; Khodabandeh et al., 2019). Freezing all layers in E1 produced weaker detection accuracy, confirming that some degree of domain-specific adaptation is necessary for KITTI's unconstrained environments.

Stage 1: Frozen Backbone

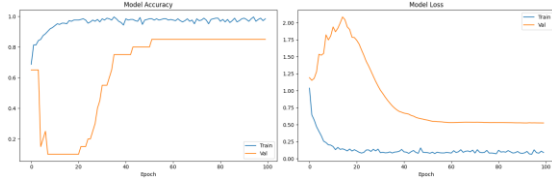


Figure 11: Learning Curve of Stage 1

In Figure 11 we noticed that the training accuracy rose quickly and stabilises near 99%, while training loss decreased steadily. However, validation accuracy stabilises near 85% and the loss remained higher than training loss. The gap reflects limited generalisation when the backbone is fully frozen as the model relies solely on ImageNet-pretrained features. These features lack the adaptation to domain-specific challenges such as illumination variation, occlusions, and object scale.

Label	Precision	Recall	F1	Support
Car	92%	100%	96%	22
Van	100%	25%	40%	4
Truck	50%	100%	67%	1
Pedestrian	0%	0%	0%	0
Cyclist	0%	0%	0%	0

Table 2: Classification Report of Stage 1

In table 2, The Car class performed well achieving 92% precision, 100% recall, and 96% F1. However, minority classes (Pedestrian, Cyclist) scored 0% across the board, this is due to the poor distribution of classes in the test set. The Van class achieved 100% precision, 25% recall and 40% F1. This suggests that the model is confusing the Car and Van due to their similarities.

Stage 2: Layer 4 & FPN unfrozen

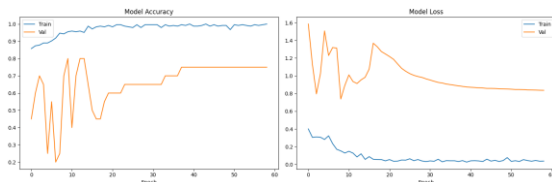


Figure 12: Learning Curves of Stage 2

In Figure 12 we noticed that training accuracy approached 100% and training

loss became very low, but validation accuracy plateaued near 70% with validation loss remaining high which is a clear sign of overfitting. Unfreezing additional layers gave the model more flexibility but also made it more sensitive to the limited training data, pushing it toward memorization rather than generalization.

Label	Precision	Recall	F1	Support
Car	88%	100%	94%	22
Van	0%	0%	0%	4
Truck	50%	100%	67%	1
Pedestrian	0%	0%	0%	0
Cyclist	0%	0%	0%	0

Table 3: Classification Report of Stage 2

In Table 3 we noticed that Car precision dipped slightly compared to Stage 1, but overall balance improved. The Car class reached an F1 of 94%, and the Truck class matched at 67%. Van, Pedestrian, and Cyclist underperformed, the results suggest that partial fine-tuning did not help the model better handle varying object appearances and scene conditions.

Stage 3: Layer 3,4 & FPN unfrozen

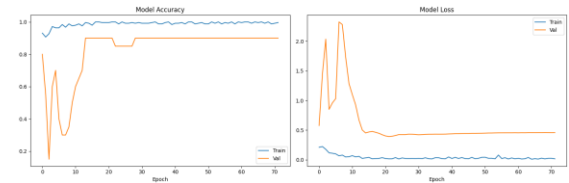


Figure 13: Learning Curves of Stage 3

In Figure 13 we notice that unfreezing Layer 4 and the Feature Pyramid Network meaningfully improved adaptation to KITTI. Validation accuracy climbed steadily to around 90%, and loss decreased more smoothly than in Stage 1. The narrower gap between training and validation curves points to better generalisation and less overfitting. Fine-tuning the higher-level semantic layers let the model retain strong low-level pretrained features while adapting deeper representations to the target domain.

Label	Precision	Recall	F1	Support
Car	92%	100%	96%	22
Van	100%	25%	40%	4
Truck	100%	100%	100%	1
Pedestrian	0%	0%	0%	0
Cyclist	0%	0%	0%	0

Table 4: Classification Report of Stage 3

In Table 4 we noticed that some minority classes improved: Van reached 100% precision and Truck achieved perfect scores, while Car's F1 edged up to 96%. These figures should be read with caution, however, given the very small support for minority classes such as Van and Truck. The stronger validation performance indicates that excessive fine-tuning improved overall robustness.

Impact of Temporal Modelling

Stage	Test Accuracy	Macro Avg	Weighted Avg
1	89%	40%	86%
2	85%	32%	79%
3	89%	40%	86%

Table 5: Comparison of Stages

The proposed Temporal FPN + Bi-LSTM architecture achieved the strongest overall performance among all evaluated configurations. Interweaving a spatial detector with convolutional LSTMs at the feature-map level successfully propagates frame-level information, significantly improving object detection accuracy over isolated single-frame baselines (Liu and Zhu, 2018; Huang et al., 2020). The findings confirm that combining foundational transfer learning with temporal sequence modelling provides a highly effective approach for autonomous driving perception tasks.

Conclusion

This project demonstrates the effectiveness of progressively deepening CNNs with regularisation for MNIST digit classification. Starting from a shallow single-block baseline

achieving 98.5% test accuracy, the systematic addition of a second convolutional block and batch normalisation (Model B) raised performance to 99.1%. Further incorporating a third convolutional block and dropout regularisation (Model C) achieved 99.3%, confirming that architectural depth combined with appropriate regularisation is a reliable strategy for improving generalisation in image classification tasks.

For Task 2, the integration of a pre-trained Faster R-CNN backbone with a Bidirectional LSTM for temporal modelling yielded an overall accuracy of 89% and a weighted mAP of 86% on the KITTI benchmark. The three-stage fine-tuning strategy demonstrated that selectively unfreezing higher-level backbone layers provides the optimal trade-off between preserving generic ImageNet representations and adapting to the KITTI driving domain.

Future Works

Several avenues of future investigation are worthwhile. For Task 1, data augmentation with random rotations, affine transformations, and elastic distortions is a well-established means of improving generalisation on MNIST, with augmented models routinely exceeding 99.6% accuracy (Simard et al., 2003). Exploring attention mechanisms such as Squeeze-and-Excitation blocks (Hu et al., 2018) could allow adaptive channel weighting with minimal parameter overhead. Investigating residual connections (He et al., 2016) would additionally enable significantly deeper architectures.

For Task 2, future work could explore more sophisticated temporal models such as Transformer-based architectures for sequence modelling, which have demonstrated strong performance in video understanding tasks. Further investigation into domain adaptation techniques and the use of additional KITTI sequences covering a wider range of environmental conditions would also be worthwhile. Real-time optimisation via model

quantisation and pruning represents a practical avenue for deployment in embedded autonomous driving systems.

References

- Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J. and Zisserman, A. (2009) ‘The Pascal Visual Object Classes (VOC) Challenge’, *International Journal of Computer Vision*, 88(2), pp. 303–338.
- Geiger, A., Lenz, P. and Urtasun, R. (2012) ‘Are we ready for autonomous driving? The KITTI vision benchmark suite’, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Providence, USA, pp. 3354–3361.
- GeeksforGeeks (2023) *Region Proposal Network (RPN) in Object Detection*. GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/computer-vision/region-proposal-network-rpn-in-object-detection/> (Accessed: 22 May 2026).
- Glorot, X. and Bengio, Y. (2010) ‘Understanding the difficulty of training deep feedforward neural networks’, in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Sardinia, Italy, pp. 249–256.
- He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep residual learning for image recognition’, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, USA, pp. 770–778.
- Hinton, G., Vinyals, O. and Dean, J. (2015) ‘Distilling the knowledge in a neural network’, *arXiv preprint arXiv:1503.02531*. Available at: <https://arxiv.org/abs/1503.02531> (Accessed: 19 May 2026).
- Hochreiter, S. and Schmidhuber, J. (1997) ‘Long short-term memory’, *Neural Computation*, 9(8), pp. 1735–1780.
- Hu, J., Shen, L. and Sun, G. (2018) ‘Squeeze-and-excitation networks’, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, USA, pp. 7132–7141.
- Huang, R., Pedoeem, J. and Chen, C. (2020) ‘YOLO-LITE: A real-time object detection algorithm optimized for non-GPU computers’, in *Proceedings of the IEEE International Conference on Big Data*, Seattle, USA, pp. 2503–2510.
- Ioffe, S. and Szegedy, C. (2015) ‘Batch normalization: Accelerating deep network training by reducing internal covariate shift’, in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, Lille, France, pp. 448–456.
- Khodabandeh, M., Vahdat, A., Ranjbar, M. and Macready, W.G. (2019) ‘A robust learning approach to domain adaptive object detection’, in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Seoul, Korea, pp. 480–490.
- Kingma, D.P. and Ba, J. (2014) ‘Adam: A method for stochastic optimization’, in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, USA. Available at: <https://arxiv.org/abs/1412.6980> (Accessed: 19 May 2026).
- Krizhevsky, A., Sutskever, I. and Hinton, G.E. (2012) ‘ImageNet classification with deep convolutional neural networks’, in *Advances in Neural Information Processing Systems (NeurIPS)*, Lake Tahoe, USA, pp. 1097–1105.
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998) ‘Gradient-based learning applied to document recognition’, *Proceedings of the IEEE*, 86(11), pp. 2278–2324.
- Lee, J. (2013) ‘Pseudo-labelling for semi-supervised image classification’, in *Proceedings of the 30th International*

Conference on Machine Learning (ICML) Workshop, Atlanta, USA.

Lin, T.Y., Dollar, P., Girshick, R., He, K., Hariharan, B. and Belongie, S. (2017) 'Feature pyramid networks for object detection', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, USA, pp. 2117–2125.

Liu, M. and Zhu, M. (2018) 'Mobile video object detection with temporally-aware feature maps', in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, USA, pp. 5686–5695.

Nair, V. and Hinton, G.E. (2010) 'Rectified linear units improve restricted Boltzmann machines', in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, Haifa, Israel, pp. 807–814.

Ren, S., He, K., Girshick, R. and Sun, J. (2015) 'Faster R-CNN: Towards real-time object detection with region proposal networks', in *Advances in Neural Information Processing Systems (NeurIPS)*, Montreal, Canada, pp. 91–99.

Simard, P.Y., Steinkraus, D. and Platt, J.C. (2003) 'Best practices for convolutional neural networks applied to visual document analysis', in *Proceedings of the 7th International Conference on Document Analysis and Recognition (ICDAR)*, Edinburgh, UK, pp. 958–963.

Simonyan, K. and Zisserman, A. (2014) 'Very deep convolutional networks for large-scale

image recognition', *arXiv preprint arXiv:1409.1556*. Available at: <https://arxiv.org/abs/1409.1556> (Accessed: 19 May 2026).

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014) 'Dropout: A simple way to prevent neural networks from overfitting', *Journal of Machine Learning Research*, 15(1), pp. 1929–1958.

Williamson, A. (2026) 'Mini-batch gradient descent for deep learning', *University of Sunderland CET3013 Lecture Notes*, Sunderland.

Wagstaff, K.L. (2026) *Realistic handwritten multi-digit writer (MDW) number recognition challenges*. *arXiv preprint arXiv:2512.00676*. doi: [arXiv](https://arxiv.org/abs/2512.00676) (Accessed: 21 May 2026).

Yara, F.B. and Gerardo, B.D. (2026) 'Regularized deep learning models for acoustic event classification', *International Journal of Advanced Trends in Computer Science and Engineering*, 15(2), pp. 36–44. doi: <https://doi.org/10.30534/ijatcse/2026/031522026>.

Yang, G-Y., Nakayama, G.K., Xiao, Z-K., Mu, T-J., Huang, X. and Hu, S-M. (2023) 'Semantic-Aware Transformation-Invariant RoI Align', *arXiv*, arXiv:2312.09609v1 [cs.CV]. Available at: <https://arxiv.org/abs/2312.09609> (Accessed: 22 May 2026).

Appendix

```
import torch

import torch.nn as nn

import torch.nn.functional as F

import torch.optim as optim

import torchvision.transforms as transforms

import pandas as pd

from torchvision.datasets import MNIST

from torch.utils.data import DataLoader, random_split


import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics import (
    confusion_matrix, classification_report,
    ConfusionMatrixDisplay
)

import copy

import warnings

warnings.filterwarnings('ignore')


SEED = 42

torch.manual_seed(SEED)

np.random.seed(SEED)


device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

print(f'Using device: {device}')

print(f'PyTorch version: {torch.__version__}')
```


BATCH_SIZE = 128

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
```

```
full_train = MNIST(root='data/', train=True, download=True, transform=transform)
full_test = MNIST(root='data/', train=False, download=True, transform=transform)
```

```
train_data, val_data = random_split(
    full_train, [42000, 18000],
    generator=torch.Generator().manual_seed(SEED)
)

val_data, test_data = random_split(
    val_data, [9000, 9000],
    generator=torch.Generator().manual_seed(SEED)
)
```

```
print(f'Train samples:    {len(train_data):,}')
print(f'Validation samples: {len(val_data):,}')
print(f'Test samples:      {len(test_data):,}')
```

```
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True,
num_workers=2, pin_memory=True)

val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False,
num_workers=2, pin_memory=True)
```

```
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False,
num_workers=2, pin_memory=True)
```

```
print(f'Train batches:    {len(train_loader)}')
```

```
print(f'Validation batches: {len(val_loader)}')
```

```
print(f'Test batches:     {len(test_loader)}')
```

```
fig, axes = plt.subplots(2, 10, figsize=(15, 3.5))
```

```
fig.suptitle('MNIST Sample Images — One Per Class (Two Rows)', fontsize=13,
fontweight='bold')
```

```
class_samples = {i: [] for i in range(10)}
```

```
for img, label in full_train:
```

```
    if len(class_samples[label]) < 2:
```

```
        class_samples[label].append(img)
```

```
    if all(len(v) == 2 for v in class_samples.values()):
```

```
        break
```

```
for col, digit in enumerate(range(10)):
```

```
    for row in range(2):
```

```
        axes[row, col].imshow(class_samples[digit][row].squeeze(), cmap='gray')
```

```
        axes[row, col].axis('off')
```

```
    if row == 0:
```

```
        axes[row, col].set_title(f'Digit {digit}', fontsize=9)
```

```
plt.tight_layout()
```

```
plt.savefig('mnist_samples.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

```
print('Fig 1: MNIST sample images — two instances per class.')
```

```

class BaselineModel(nn.Module):

    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(6272, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)

```

```

class BaselineModelV2(nn.Module):

    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(3136, 256)
        self.bn3 = nn.BatchNorm1d(256)
        self.fc2 = nn.Linear(256, 10)

```

```

def forward(self, x):

    x = self.pool(F.relu(self.bn1(self.conv1(x))))

    x = self.pool(F.relu(self.bn2(self.conv2(x))))

    x = x.view(x.size(0), -1)

    x = F.relu(self.bn3(self.fc1(x)))

    return self.fc2(x)

```

```

class FineTunedModel(nn.Module):

```

```

    def __init__(self, dropout_rate=0.4):

        super().__init__()

        self.conv1a = nn.Conv2d(1, 32, kernel_size=3, padding=1)

        self.bn1a = nn.BatchNorm2d(32)

        self.conv1b = nn.Conv2d(32, 32, kernel_size=3, padding=1)

        self.bn1b = nn.BatchNorm2d(32)

        self.conv2a = nn.Conv2d(32, 64, kernel_size=3, padding=1)

        self.bn2a = nn.BatchNorm2d(64)

        self.conv2b = nn.Conv2d(64, 64, kernel_size=3, padding=1)

        self.bn2b = nn.BatchNorm2d(64)

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)

        self.bn3 = nn.BatchNorm2d(128)

        self.pool = nn.MaxPool2d(2, 2)

        self.dropout = nn.Dropout(dropout_rate)

        self.fc1 = nn.Linear(1152, 256)

        self.bn4 = nn.BatchNorm1d(256)

        self.fc2 = nn.Linear(256, 10)

```

```

    def forward(self, x):

```

```
x = F.relu(self.bn1a(self.conv1a(x)))
x = self.pool(F.relu(self.bn1b(self.conv1b(x))))
```

```
x = F.relu(self.bn2a(self.conv2a(x)))
x = self.pool(F.relu(self.bn2b(self.conv2b(x))))
```

```
x = self.pool(F.relu(self.bn3(self.conv3(x))))
```

```
x = x.view(x.size(0), -1)
x = self.dropout(x)
x = F.relu(self.bn4(self.fc1(x)))
x = self.dropout(x)
return self.fc2(x)
```

```
history = {
    'train_loss': [], 'val_loss': [],
    'train_acc': [], 'val_acc': []
}
```

```
def train_model(model, train_loader, val_loader,
                epochs=25, lr=1e-3,
                patience=7, model_name='model'):
```

```
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
```

```
optimizer, mode='min', factor=0.5, patience=3,  
)
```

```
best_val_loss = float('inf')
```

```
best_state = None
```

```
no_improve = 0
```

```
for epoch in range(epochs):
```

```
    model.train()
```

```
    running_loss, correct, total = 0.0, 0, 0
```

```
    for images, labels in train_loader:
```

```
        images, labels = images.to(device), labels.to(device)
```

```
        optimizer.zero_grad()
```

```
        outputs = model(images)
```

```
        loss = criterion(outputs, labels)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
        running_loss += loss.item() * images.size(0)
```

```
        _, preds = torch.max(outputs, 1)
```

```
        correct += (preds == labels).sum().item()
```

```
        total += labels.size(0)
```

```
    train_loss = running_loss / total
```

```
    train_acc = correct / total
```

```
    model.eval()
```

```
    val_loss, val_correct, val_total = 0.0, 0, 0
```

```
    with torch.no_grad():
```

```
        for images, labels in val_loader:
```

```

images, labels = images.to(device), labels.to(device)

outputs = model(images)

loss = criterion(outputs, labels)

val_loss += loss.item() * images.size(0)

_, preds = torch.max(outputs, 1)

val_correct += (preds == labels).sum().item()

val_total += labels.size(0)


val_loss /= val_total

val_acc = val_correct / val_total


history['train_loss'].append(train_loss)
history['val_loss'].append(val_loss)
history['train_acc'].append(train_acc)
history['val_acc'].append(val_acc)


scheduler.step(val_loss)


if val_loss < best_val_loss:
    best_val_loss = val_loss

    best_state = copy.deepcopy(model.state_dict())

    no_improve = 0
else:
    no_improve += 1


if (epoch + 1) % 5 == 0 or epoch == 0:
    print(f'[{model_name}] Epoch {epoch+1:3d}/{epochs} | '
          f'Train Loss: {train_loss:.4f} Acc: {train_acc:.4f} | '
          f'Val Loss: {val_loss:.4f} Acc: {val_acc:.4f}')

```



```
if no_improve >= patience:
    print(f[{model_name}] Early stopping at epoch {epoch+1}.')
    break
```

```
print(f[{model_name}] Training complete. '
      fBest val loss: {best_val_loss:.4f} | ')
```

```
model.load_state_dict(best_state) # restore best weights
return history, model
```

```
def evaluate_model(model, test_loader):
    model.eval()
    correct, total = 0, 0
    all_preds, all_labels = [], []

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
```

```
accuracy = correct / total  
return accuracy, np.array(all_preds), np.array(all_labels)
```

```
model_a = BaselineModel()  
print(model_a)
```

```
history_a, model_a = train_model(  
    model_a, train_loader, val_loader,  
    epochs=25, lr=1e-3, patience=7,  
    model_name='Baseline'  
)
```

```
acc_a, preds_a, labels_a = evaluate_model(model_a, test_loader)  
print(f'\nModel A — Test Accuracy: {acc_a:.4f} ({acc_a*100:.2f}%)')
```

```
torch.save(model_a.state_dict(), "../src/Baseline_Model")
```

```
model_b = BaselineModelV2()  
print(model_b)
```

```
history_b, model_b = train_model(  
    model_b, train_loader, val_loader,  
    epochs=25, lr=1e-3, patience=7,  
    model_name='BaselineV2'
```

)

```
acc_b, preds_b, labels_b = evaluate_model(model_b, test_loader)
print(f'\nModel B — Test Accuracy: {acc_b:.4f} ({acc_b*100:.2f}%)')
```

```
model_c = FineTunedModel(dropout_rate=0.4)
print(model_c)
```

```
history_c, model_c = train_model(
    model_c, train_loader, val_loader,
    epochs=25, lr=1e-3, patience=7,
    model_name='Fined Tuned'
)
```

```
acc_c, preds_c, labels_c = evaluate_model(model_c, test_loader)
print(f'\nModel C — Test Accuracy: {acc_c:.4f} ({acc_c*100:.2f}%)')
```

```
fig, axes = plt.subplots(2, 3, figsize=(16, 9))
fig.suptitle('Learning Curves', fontsize=14, fontweight='bold')
```

```
model_configs = [
    ('Model A', history_a, '#2196F3'),
    ('Model B', history_b, '#4CAF50'),
    ('Model C', history_c, '#FF5722'),
]
```

```

for col, (name, hist, colour) in enumerate(model_configs):

    epochs = range(1, len(hist['train_loss']) + 1)

    # Loss
    axes[0, col].plot(epochs, hist['train_loss'],
                      color=colour, label='Train', linewidth=2)
    axes[0, col].plot(epochs, hist['val_loss'], color=colour,
                      ls='--', label='Val', linewidth=2, alpha=0.8)
    axes[0, col].set_title(f'{name}\nLoss', fontsize=11)
    axes[0, col].set_xlabel('Epoch')
    axes[0, col].set_ylabel('Cross-Entropy Loss')
    axes[0, col].legend()
    axes[0, col].grid(True, alpha=0.3)

    # Accuracy
    axes[1, col].plot(epochs, [a*100 for a in hist['train_acc']],
                      color=colour, label='Train', linewidth=2)
    axes[1, col].plot(epochs, [a*100 for a in hist['val_acc']],
                      color=colour, ls='--', label='Val', linewidth=2, alpha=0.8)
    axes[1, col].set_title(f'{name}\nAccuracy', fontsize=11)
    axes[1, col].set_xlabel('Epoch')
    axes[1, col].set_ylabel('Accuracy (%)')
    axes[1, col].set_ylim([85, 100])
    axes[1, col].legend()
    axes[1, col].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('../docs/Visualisations/learning_curves.png', dpi=150, bbox_inches='tight')
plt.show()

```

```
print('Fig 2: Learning curves (loss and accuracy) for all three models.')
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
```

```
fig.suptitle('Confusion Matrices on Test Set', fontsize=14, fontweight='bold')
```

```
model_results = [
```

```
    ('Model A', preds_a, labels_a),
```

```
    ('Model B', preds_b, labels_b),
```

```
    ('Model C', preds_c, labels_c),
```

```
]
```

```
for ax, (name, preds, labels) in zip(axes, model_results):
```

```
    cm = confusion_matrix(labels, preds)
```

```
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
```

```
                                display_labels=list(range(10)))
```

```
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
```

```
    ax.set_title(name, fontsize=12)
```

```
plt.tight_layout()
```

```
plt.savefig('../docs/Visualisations/confusion_matrices.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

```
print('Fig 3: Confusion matrices for Models A, B, and C.')
```

```
class_names = [str(i) for i in range(10)]
```

```
for name, preds, labels in [('Model A (Baseline)', preds_a, labels_a),
```

```
                            ('Model B (Medium)', preds_b, labels_b),
```

```

        ('Model C (Deep)', preds_c, labels_c)]:

print(f'\n{"—"*55}')

print(f' {name} — Classification Report')

print(f'{"—"*55}')

print(classification_report(labels, preds, target_names=class_names))


summary = pd.DataFrame({
    'Model':      ['A (Baseline)', 'B (Medium)', 'C (Deep)'],
    'Conv Blocks': [1, 2, 3],
    'Batch Norm':  ['No', 'Yes', 'Yes'],
    'Dropout':     ['No', 'No', 'Yes (0.4)'],
    'Best Val Acc (%)': [
        round(max(history_a['val_acc']) * 100, 2),
        round(max(history_b['val_acc']) * 100, 2),
        round(max(history_c['val_acc']) * 100, 2),
    ],
    'Test Acc (%)': [round(acc_a*100, 2), round(acc_b*100, 2), round(acc_c*100, 2)],
    'Epochs Trained': [
        len(history_a['train_loss']),
        len(history_b['train_loss']),
        len(history_c['train_loss']),
    ]
})

print('\n=== Summary Comparison Table ===')

print(summary.to_string(index=False))


import os

```

```
import cv2
import numpy as np
import pandas as pd
import torch
import torchvision.transforms as T
from torch.utils.data import Dataset, DataLoader
from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchmetrics.detection.mean_ap import MeanAveragePrecision
from PIL import Image
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from collections import Counter
from sklearn.preprocessing import label_binarize
import xml.etree.ElementTree as ET
import torch
from sklearn.metrics import (average_precision_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)
from pprint import pprint
import matplotlib.pyplot as plt
```

```
TARGET_FRAMES = 32
```

```
COORD_FEATURES = 9
```

```
VISUAL_FEATURES = 256
```

```
NUM_CLASSES = 5
```

```
CLASS_MAP = {'Car': 0, 'Van': 1, 'Truck': 2, 'Pedestrian': 3, 'Cyclist': 4}
```

```
INV_CLASS_MAP = {v: k for k, v in CLASS_MAP.items()}
```

SEED = 42

torch.manual_seed(SEED)

np.random.seed(SEED)

```
ALL_VIDEO_SEQ = [("../data/Video_1/Video.xml",
                  "../data/Video_1/calib_cam_to_cam.txt",
                  "../data/Video_1/calib_imu_to_velo.txt",
                  "../data/Video_1/calib_velo_to_cam.txt", "../data/Video_1/sequence_images/",
                  "Video_1"), ("../data/Video_2/Video.xml",
                              "../data/Video_2/calib_cam_to_cam.txt",
                              "../data/Video_2/calib_imu_to_velo.txt",
                              "../data/Video_2/calib_velo_to_cam.txt", "../data/Video_2/sequence_images/", "Video_2"),
                              ("../data/Video_3/Video.xml",
                              "../data/Video_3/calib_cam_to_cam.txt",
                              "../data/Video_3/calib_imu_to_velo.txt",
                              "../data/Video_3/calib_velo_to_cam.txt", "../data/Video_3/sequence_images/",
                              "Video_3"),
                              ("../data/Video_4/Video.xml",
                              "../data/Video_4/calib_cam_to_cam.txt",
                              "../data/Video_4/calib_imu_to_velo.txt",
                              "../data/Video_4/calib_velo_to_cam.txt", "../data/Video_4/sequence_images/",
                              "Video_4"),
                              ("../data/Video_9/Video.xml",
                              "../data/Video_9/calib_cam_to_cam.txt",
                              "../data/Video_9/calib_imu_to_velo.txt",
                              "../data/Video_9/calib_velo_to_cam.txt", "../data/Video_9/sequence_images/",
                              "Video_9")]
```



```
("../data/Video_10/Video.xml",  
    "../data/Video_10/calib_cam_to_cam.txt",  
    "../data/Video_10/calib_imu_to_velo.txt",  
    "../data/Video_10/calib_velo_to_cam.txt", "../data/Video_10/sequence_images/",  
    "Video_10"),
```

```
("../data/Video_11/Video.xml",  
    "../data/Video_11/calib_cam_to_cam.txt",  
    "../data/Video_11/calib_imu_to_velo.txt",  
    "../data/Video_11/calib_velo_to_cam.txt", "../data/Video_11/sequence_images/",  
    "Video_11"),
```

```
("../data/Video_12/Video.xml",  
    "../data/Video_12/calib_cam_to_cam.txt",  
    "../data/Video_12/calib_imu_to_velo.txt",  
    "../data/Video_12/calib_velo_to_cam.txt", "../data/Video_12/sequence_images/",  
    "Video_12"),
```

```
("../data/Video_13/Video.xml",  
    "../data/Video_13/calib_cam_to_cam.txt",  
    "../data/Video_13/calib_imu_to_velo.txt",  
    "../data/Video_13/calib_velo_to_cam.txt", "../data/Video_13/sequence_images/",  
    "Video_13"),
```

```
]
```

```
train_set = ALL_VIDEO_SEQ[0:7]
```

```
val_set = ALL_VIDEO_SEQ[7:8]
```

```
test_set = ALL_VIDEO_SEQ[8:9]
```

```
print(f'Train sequences : {len(train_set)}')
```

```
print(f'Val sequences : {len(val_set)}')
```

```
print(f'Test sequences : {len(test_set)}')
```

```
def check_data_paths():
```

```
    all_ok = True
```

```
    for xml_path, cam_to_cam, imu_to_velo, velo_to_cam, images_dir, name in  
ALL_VIDEO_SEQ:
```

```
        print(xml_path, cam_to_cam, imu_to_velo, velo_to_cam, images_dir, name)
```

```
        video_name = name
```

```
        img_dir = images_dir
```

```
        xml_ok = os.path.isfile(xml_path)
```

```
        c2c_ok = os.path.isfile(cam_to_cam)
```

```
        i2v_ok = os.path.isfile(imu_to_velo)
```

```
        v2c_ok = os.path.isfile(velo_to_cam)
```

```
        img_ok = os.path.isdir(img_dir)
```

```
        n_images = len([f for f in os.listdir(img_dir) if f.endswith('.png')]) if img_ok else 0
```

```
        status = '✓' if all([xml_ok, c2c_ok, i2v_ok, v2c_ok, img_ok]) else 'X'
```

```
        if status == 'X':
```

```
            all_ok = False
```

```
        print(f'{status} {video_name} | xml={xml_ok} c2c={c2c_ok} '
```

```
            fi2v={i2v_ok} v2c={v2c_ok} imgs={n_images} ({img_dir})')
```

```

if all_ok:
    print('\nAll paths verified')
else:
    print('\n One or more paths are missing')

```

```

check_data_paths()

```

```

def _parse_calib_file(path):
    calib = {}
    with open(path, 'r') as f:
        for line in f:
            line = line.strip()
            if not line or ':' not in line:
                continue
            key, vals = line.split(':', 1)
            try:
                calib[key.strip()] = np.array([float(v) for v in vals.split()])
            except ValueError:
                pass
    return calib

```

```

def get_projection_matrix(cam_to_cam_path, velo_to_cam_path):
    vc = _parse_calib_file(velo_to_cam_path)
    R_vc = vc['R'].reshape(3, 3)
    T_vc = vc['T'].reshape(3, 1)
    tr_velo_to_cam = np.vstack([np.hstack([R_vc, T_vc]), [0, 0, 0, 1]])

```

```

cc = _parse_calib_file(cam_to_cam_path)
r_rect = np.eye(4)
r_rect[:3, :3] = cc['R_rect_00'].reshape(3, 3)
p_rect_02 = cc['P_rect_02'].reshape(3, 4)

return p_rect_02 @ r_rect @ tr_velo_to_cam

```

```

def project_3d_to_2d(p_matrix, x, y, z):
    pt_3d = np.array([x, y, z, 1.0])
    pt_2d = p_matrix @ pt_3d
    if pt_2d[2] == 0:
        return 0, 0
    return int(round(pt_2d[0] / pt_2d[2])), int(round(pt_2d[1] / pt_2d[2]))

```

```

def compute_iou(box1, box2):
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    intersection = max(0, x2 - x1) * max(0, y2 - y1)

    area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
    area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])
    union = area1 + area2 - intersection

    return intersection / union if union > 0 else 0.0

```

```

def compute_iou_batch(pred_boxes, gt_boxes):

    x1 = np.maximum(pred_boxes[:, 0], gt_boxes[:, 0])
    y1 = np.maximum(pred_boxes[:, 1], gt_boxes[:, 1])
    x2 = np.minimum(pred_boxes[:, 2], gt_boxes[:, 2])
    y2 = np.minimum(pred_boxes[:, 3], gt_boxes[:, 3])

    intersection = np.maximum(0, x2 - x1) * np.maximum(0, y2 - y1)

    area1 = (pred_boxes[:, 2] - pred_boxes[:, 0]) * (pred_boxes[:, 3] - pred_boxes[:, 1])
    area2 = (gt_boxes[:, 2] - gt_boxes[:, 0]) * (gt_boxes[:, 3] - gt_boxes[:, 1])
    union = area1 + area2 - intersection

    return np.where(union > 0, intersection / union, 0.0)

def evaluate_iou(pred_boxes, gt_boxes, threshold=0.5, label=""):

    ious = compute_iou_batch(pred_boxes, gt_boxes)
    mean_iou = np.mean(ious)
    detection_rate = np.mean(ious >= threshold)
    tag = f'[{label}] ' if label else "
    print(f'{tag}Mean IoU      : {mean_iou:.4f}')
    print(f'{tag}Detection Rate : {detection_rate:.4f} (threshold={threshold})')
    return ious

```

```

class VisionExtractor:

```

```

def __init__(self):
    full_model = fasterrcnn_resnet50_fpn(pretrained=True)
    self.backbone = full_model.backbone
    self.backbone.eval()
    self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    self.backbone.to(self.device)
    with torch.no_grad():
        dummy = torch.zeros(1, 3, 224, 224).to(self.device)
        self.feature_dim = self.backbone(dummy)['0'].shape[1]
    self.transform = T.Compose([
        T.Resize((224, 224)), T.ToTensor(),
        T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

```

```

def extract(self, pil_img):
    img_t = self.transform(pil_img).unsqueeze(0).to(self.device)
    with torch.no_grad():
        feats = self.backbone(img_t)
    return torch.mean(feats['0'], dim=[2, 3]).cpu().numpy().flatten()

```

```

def freeze_backbone_stage1(extractor):
    for param in extractor.backbone.parameters():
        param.requires_grad = False

trainable = sum(p.numel() for p in extractor.backbone.parameters() if p.requires_grad)
total     = sum(p.numel() for p in extractor.backbone.parameters())
print(f'[Stage 1] Backbone frozen — trainable backbone params: {trainable:,} / {total:,}')

```

```

def unfreeze_stage2(extractor):
    for param in extractor.backbone.parameters():
        param.requires_grad = False

    for name, param in extractor.backbone.named_parameters():
        if 'layer4' in name or 'fpn' in name:
            param.requires_grad = True

    trainable = sum(p.numel() for p in extractor.backbone.parameters() if p.requires_grad)
    total     = sum(p.numel() for p in extractor.backbone.parameters())

    print(f'[Stage 2] Unfrozen layer4 + FPN — trainable backbone params: {trainable:,} / {total:,}')

```

```

def unfreeze_stage3(extractor):
    for param in extractor.backbone.parameters():
        param.requires_grad = False

    for name, param in extractor.backbone.named_parameters():
        if any(block in name for block in ['layer3', 'layer4', 'fpn']):
            param.requires_grad = True

    trainable = sum(p.numel() for p in extractor.backbone.parameters() if p.requires_grad)
    total     = sum(p.numel() for p in extractor.backbone.parameters())

    print(f'[Stage 3] Unfrozen layer3 + layer4 + FPN — trainable backbone params: {trainable:,} / {total:,}')

```

```

def compute_2d_bbox_from_3d(p_matrix, tx, ty, tz, w, h, l):

```

```
half_w = w / 2.0
```

```
half_l = l / 2.0
```

```
corners_3d = [
```

```
    (tx + dw, ty + dl, tz + dh)
```

```
    for dw in (-half_w, half_w)
```

```
    for dl in (-half_l, half_l)
```

```
    for dh in (0, h)
```

```
]
```

```
us, vs = [], []
```

```
for cx, cy, cz in corners_3d:
```

```
    u, v = project_3d_to_2d(p_matrix, cx, cy, cz)
```

```
    us.append(u)
```

```
    vs.append(v)
```

```
return min(us), min(vs), max(us), max(vs)
```

```
class KITTIDataSet(Dataset):
```

```
    def __init__(self, sequences, extractor, set, augment=False,):
```

```
        self.sequences = sequences
```

```
        self.extractor = extractor
```

```
        self.set = set
```

```
        self.augment = augment
```

```
        self.X, self.y = self._build()
```

```
    def __len__(self):
```



```
return len(self.y)
```

```
def __getitem__(self):
```

```
    pass
```

```
def _build(self):
```

```
    all_X, all_y = [], []
```

```
    for xml_path, cam_to_cam, _, velo_to_cam, images_dir, name in self.sequences:
```

```
        p_mat = get_projection_matrix(cam_to_cam, velo_to_cam)
```

```
        tracklets = self._parse_xml(xml_path)
```

```
        for label, frames in tracklets:
```

```
            if label not in CLASS_MAP:
```

```
                continue
```

```
            seq = self._build_sequence(frames, images_dir)
```

```
            all_X.append(seq)
```

```
            all_y.append(CLASS_MAP[label])
```

```
all_X = np.array(all_X, dtype=np.float32)
```

```
all_y = np.array(all_y, dtype=np.int64)
```

```
np.save(f'../data/numpy_files/all_X_{self.set}.npz', all_X)
```

```
np.save(f'../data/numpy_files/all_y_{self.set}.npz', all_y)
```

```
return all_X, all_y
```

```

def _parse_xml(self, xml_path):
    tree = ET.parse(xml_path)
    root = tree.getroot()
    tracklets = []
    for obj in root.findall('./item'):
        label = obj.findtext('objectType', default="").strip()
        frames = []
        for frame_idx, pose in enumerate(obj.findall('./item')):
            tx = float(pose.findtext('tx', '0'))
            ty = float(pose.findtext('ty', '0'))
            tz = float(pose.findtext('tz', '0'))
            rx = float(pose.findtext('rx', '0'))
            ry = float(pose.findtext('ry', '0'))
            rz = float(pose.findtext('rz', '0'))
            w = float(obj.findtext('w', '0'))
            h = float(obj.findtext('h', '0'))
            l = float(obj.findtext('l', '0'))
            frames.append(dict(tx=tx, ty=ty, tz=tz,
                               rx=rx, ry=ry, rz=rz,
                               w=w, h=h, l=l,
                               frame_idx=frame_idx))
        if frames:
            tracklets.append((label, frames))
    return tracklets

```

```

def _build_sequence(self, frames, images_dir):

```

```

if len(frames) >= TARGET_FRAMES:
    if self.augment:
        start = np.random.randint(0, len(frames) - TARGET_FRAMES + 1)
    else:
        start = 0
    frames = frames[start: start + TARGET_FRAMES]
else:
    pad = [frames[-1]] * (TARGET_FRAMES - len(frames))
    frames = frames + pad

seq = []
for f in frames:
    coord = np.array([f['tx'], f['ty'], f['tz'],
                     f['rx'], f['ry'], f['rz'],
                     f['w'], f['h'], f['l']], dtype=np.float32)
    img_path = os.path.join(images_dir, f'{f["frame_idx"]:010d}.png')
    if os.path.exists(img_path):
        pil_img = Image.open(img_path).convert('RGB')
        vis = self.extractor.extract(pil_img).astype(np.float32)
    else:
        vis = np.zeros(self.extractor.feature_dim, dtype=np.float32)
    seq.append(np.concatenate([coord, vis]))

return np.stack(seq)

```

```

def make_datasets(extractor):

```

```

print('Building TRAIN dataset ...')

train_ds = KITTIIDataset(train_set, extractor, augment=True, set="train")

print('Building VAL dataset ...')

val_ds = KITTIIDataset(val_set, extractor, augment=False, set="validation")

print('Building TEST dataset ...')

test_ds = KITTIIDataset(test_set, extractor, augment=False, set="test")

print(f' train={len(train_ds)} val={len(val_ds)} test={len(test_ds)} tracklets')

return train_ds, val_ds, test_ds

```

```

class TemporalFPNBlock(layers.Layer):

    def __init__(self, out_channels: int = 128, **kwargs):
        super().__init__(**kwargs)

        self.out_channels = out_channels

        self.p3_conv = layers.Conv1D(out_channels, 3, padding='same',
                                       dilation_rate=1, activation='relu', name='p3_conv')

        self.p4_conv = layers.Conv1D(out_channels, 3, padding='same',
                                       dilation_rate=2, activation='relu', name='p4_conv')

        self.p5_conv = layers.Conv1D(out_channels, 3, padding='same',
                                       dilation_rate=4, activation='relu', name='p5_conv')

        self.lat_p4 = layers.Conv1D(out_channels, 1, padding='same', name='lat_p4')

```

```
self.lat_p3 = layers.Conv1D(out_channels, 1, padding='same', name='lat_p3')
```

```
self.merge_p4 = layers.Conv1D(out_channels, 3, padding='same',  
                                activation='relu', name='merge_p4')
```

```
self.merge_p3 = layers.Conv1D(out_channels, 3, padding='same',  
                                activation='relu', name='merge_p3')
```

```
self.out_proj = layers.Conv1D(out_channels, 1, padding='same', name='fpn_out')
```

```
self.bn_out = layers.BatchNormalization(name='fpn_bn')
```

```
def call(self, x, training=False):
```

```
    p3 = self.p3_conv(x)
```

```
    p4 = self.p4_conv(x)
```

```
    p5 = self.p5_conv(x)
```

```
    p4_td = self.merge_p4(self.lat_p4(p4) + p5)
```

```
    p3_td = self.merge_p3(self.lat_p3(p3) + p4_td)
```

```
    out = self.bn_out(self.out_proj(p3_td), training=training)
```

```
    return out
```

```
def get_config(self):
```

```
    cfg = super().get_config()
```

```
    cfg.update({'out_channels': self.out_channels})
```

```
    return cfg
```

```

def plot_training_metrics(history):
    fig, ax = plt.subplots(1, 2, figsize=(15, 5))
    ax[0].plot(history.history['accuracy'], label='Train')
    ax[0].plot(history.history['val_accuracy'], label='Val')
    ax[0].set_title('Model Accuracy')
    ax[0].set_xlabel('Epoch'); ax[0].legend()

    ax[1].plot(history.history['loss'], label='Train')
    ax[1].plot(history.history['val_loss'], label='Val')
    ax[1].set_title('Model Loss')
    ax[1].set_xlabel('Epoch'); ax[1].legend()

    plt.tight_layout(); plt.show()

def plot_cm(y_true, y_pred, title='Confusion Matrix'):
    y_true_labels = np.array(y_true).flatten().astype(int)
    y_pred_labels = np.argmax(y_pred, axis=1)
    cm = confusion_matrix(y_true_labels, y_pred_labels,
                          labels=list(CLASS_MAP.values()))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                                  display_labels=list(CLASS_MAP.keys()))
    disp.plot(cmap='Blues')
    plt.title(title)
    plt.tight_layout(); plt.show()

def compute_2d_bbox_from_3d(p_matrix, tx, ty, tz, w, h, l):

```

```
half_w = w / 2.0
```

```
half_l = l / 2.0
```

```
corners_3d = [
```

```
    (tx + dw, ty + dl, tz + dh)
```

```
    for dw in (-half_w, half_w)
```

```
    for dl in (-half_l, half_l)
```

```
    for dh in (0, h)
```

```
]
```

```
us, vs = [], []
```

```
for cx, cy, cz in corners_3d:
```

```
    u, v = project_3d_to_2d(p_matrix, cx, cy, cz)
```

```
    us.append(u)
```

```
    vs.append(v)
```

```
return min(us), min(vs), max(us), max(vs)
```

```
def visualize_inference(img_path, p_matrix, tx, ty, tz, w, h, l,
```

```
    pred_idx, true_idx):
```

```
    img = cv2.imread(img_path)
```

```
    if img is None:
```

```
        print(f"Image not found: {img_path}")
```

```
        return
```

```
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
    img_h, img_w = img.shape[:2]
```

```
u_min, v_min, u_max, v_max = compute_2d_bbox_from_3d(
    p_matrix, tx, ty, tz, w, h, l
)
```

```
u_min = max(0, min(u_min, img_w - 1))
v_min = max(0, min(v_min, img_h - 1))
u_max = max(0, min(u_max, img_w - 1))
v_max = max(0, min(v_max, img_h - 1))
```

```
color = (0, 255, 0) if pred_idx == true_idx else (255, 0, 0)
```

```
cv2.rectangle(img, (u_min, v_min), (u_max, v_max), color, 2)
```

```
label_text = (f'Pred: {INV_CLASS_MAP[pred_idx]} | '
              f'True: {INV_CLASS_MAP[true_idx]}')
label_y = max(v_min - 10, 15)      # keep text inside the frame
cv2.putText(img, label_text, (u_min, label_y),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
```

```
plt.figure(figsize=(12, 7))
plt.imshow(img)
plt.axis("off")
plt.tight_layout()
plt.show()
```

```
extractor = VisionExtractor()
```

```
#train_ds, val_ds, test_ds, train_loader, val_loader, test_loader = make_datasets(extractor)
```



```

# load numpy files

X_train = np.load(file="../data/numpy_files/all_X_train.npy") # Videos 9 & 10
y_train = np.load(file="../data/numpy_files/all_y_train.npy")
X_val = np.load(file="../data/numpy_files/all_X_valdation.npy")
y_val = np.load(file="../data/numpy_files/all_y_valdation.npy")
X_test = np.load(file="../data/numpy_files/all_X_test.npy")
y_test = np.load(file="../data/numpy_files/all_y_test.npy")


print(f'X_train: {X_train.shape} y_train: {y_train.shape}')
print(f'X_val : {X_val.shape} y_val : {y_val.shape}')
print(f'X_test : {X_test.shape} y_test : {y_test.shape}')


def build_model():

    inp = layers.Input(shape=(TARGET_FRAMES, COORD_FEATURES +
    VISUAL_FEATURES),

                        name='sequence_input')

    x = layers.Conv1D(128, kernel_size=3, padding='same',
                      activation='relu', name='prefix_conv')(inp)
    x = layers.BatchNormalization(name='prefix_bn')(x)

    x = TemporalFPNBlock(out_channels=128, name='temporal_fpn')(x)

    x = layers.Bidirectional(layers.LSTM(128, return_sequences=True),
                             name='bilstm_1')(x)

```

```
x = layers.Bidirectional(layers.LSTM(64), name='bilstm_2')(x)
```

```
x = layers.Dense(128, activation='relu', name='fc_head')(x)
```

```
x = layers.Dropout(0.5, name='dropout')(x)
```

```
out = layers.Dense(NUM_CLASSES, activation='softmax', name='predictions')(x)
```

```
model = models.Model(inp, out, name='3D-ResNet-Temporal-Stacking-LSTM')
```

```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

```
return model
```

```
print("Building Model")
```

```
model = build_model()
```

```
model.summary()
```

```
STAGE_CONFIG = [  
    {'stage': 1, 'fn': freeze_backbone_stage1, 'lr': 1e-3, 'epochs': 100,  
     'desc': 'Frozen backbone — head only'},  
    {'stage': 2, 'fn': unfreeze_stage2, 'lr': 1e-3, 'epochs': 100,  
     'desc': 'layer4 + FPN unfrozen'},  
    {'stage': 3, 'fn': unfreeze_stage3, 'lr': 1e-3, 'epochs': 100,  
     'desc': 'layer3 + layer4 + FPN unfrozen'},  
]
```

```
fine_tune_histories = {}
```

```
for cfg in STAGE_CONFIG:
```

```
    print(f'\n{'='*60}')
```

```
    print(f' Fine-Tuning Stage {cfg['stage']}: {cfg['desc']}')
```

```
    print(f' Learning rate: {cfg['lr']} | Epochs: {cfg['epochs']}')
```

```
    print(f'\n{'='*60}')
```

```
    cfg['fn'](extractor)
```

```
    extractor.backbone.train()
```

```
    stage_cb = [
```

```
        callbacks.EarlyStopping(monitor='val_loss', patience=50,
```

```
                                restore_best_weights=True, verbose=1),
```

```
        callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,
```

```
                                    patience=3, min_lr=1e-7, verbose=1),
```

```
        callbacks.ModelCheckpoint(
```

```
            f'best_model_stage{cfg["stage"]}.keras',
```

```
            monitor='val_loss', save_best_only=True, verbose=1),
```

```
    ]
```

```
    model.compile(
```

```
        optimizer=tf.keras.optimizers.Adam(learning_rate=cfg['lr']),
```

```
        loss='sparse_categorical_crossentropy',
```

```
        metrics=['accuracy']
```

```
    )
```

```
    hist = model.fit(
```

```
        X_train, y_train,
```

```

validation_data=(X_val, y_val),
epochs=cfg['epochs'],
batch_size=32,
callbacks=stage_cb,
verbose=1,
)
fine_tune_histories[f'stage{cfg["stage"]}'] = hist
plot_training_metrics(hist)
print('\n—— Test split (Video 13) ——')
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f' Loss: {test_loss:.4f} Accuracy: {test_acc:.4f}')

y_test_pred = model.predict(X_test)
plot_cm(y_test, y_test_pred, title='Confusion Matrix — Test Split')
print(classification_report(
y_test,
np.argmax(y_test_pred, axis=1),
labels=list(CLASS_MAP.values()),
target_names=list(CLASS_MAP.keys()))))

print('\nThree-stage fine-tuning complete.')

y_test_bin = label_binarize(y_test, classes=list(CLASS_MAP.values()))

ap_per_class = {}
for cls_idx, cls_name in INV_CLASS_MAP.items():

```

```

ap = average_precision_score(y_test_bin[:, cls_idx], y_test_pred[:, cls_idx])
ap_per_class[cls_name] = ap
print(f' AP {cls_name:<12}: {ap:.4f}')

support = Counter(y_test.tolist())
total = sum(support.values())
weighted_map = sum(ap_per_class[INV_CLASS_MAP[c]] * support[c] / total
                    for c in range(NUM_CLASSES))
print(f'\nWeighted mAP: {weighted_map:.4f}')

```

```

SAMPLE_IDX = 0

```

```

xml_path, cam_to_cam, _, velo_to_cam, images_dir, video_name = test_set[0]
p_matrix = get_projection_matrix(cam_to_cam, velo_to_cam)

```

```

pred_idx = int(np.argmax(y_test_pred[SAMPLE_IDX]))
true_idx = int(y_test[SAMPLE_IDX])

```

```

frame_idx = 0
img_path = os.path.join(images_dir, f'{frame_idx:010d}.png')

```

```

tx, ty, tz, w, h, l = X_test[SAMPLE_IDX, 0, 0], X_test[SAMPLE_IDX, 0, 1],
X_test[SAMPLE_IDX, 0, 2], X_test[SAMPLE_IDX, 0, 6],
X_test[SAMPLE_IDX, 0, 7], X_test[SAMPLE_IDX, 0, 8]

```

```

visualize_inference(img_path, p_matrix, tx, ty, tz, w, h, l, pred_idx, true_idx)

```

